



# ARA0110 PADRÕES DE PROJETO

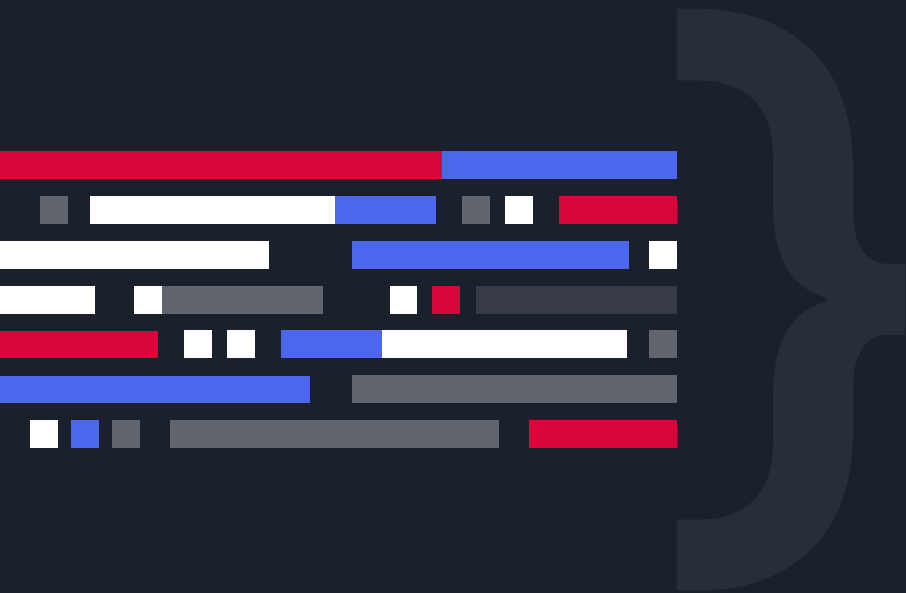
Aula 05: Padrões de projeto estruturais - parte 2





RECAPITULANDO





## PADRÕES ESTRUTURAIS

Os padrões de projeto estruturais focam na forma como a estrutura dos objetos são compostas, para formar estruturas maiores e completas.

Nesses padrões, encontramos diversas abordagens de estruturação e composição de objetos.



# DECORATOR





## DEFINIÇÃO

Dinamicamente, agregar responsabilidades adicionais a um objeto. Os decorators fornecem uma alternativa flexível ao uso de subclasses para extensão de funcionalidades.

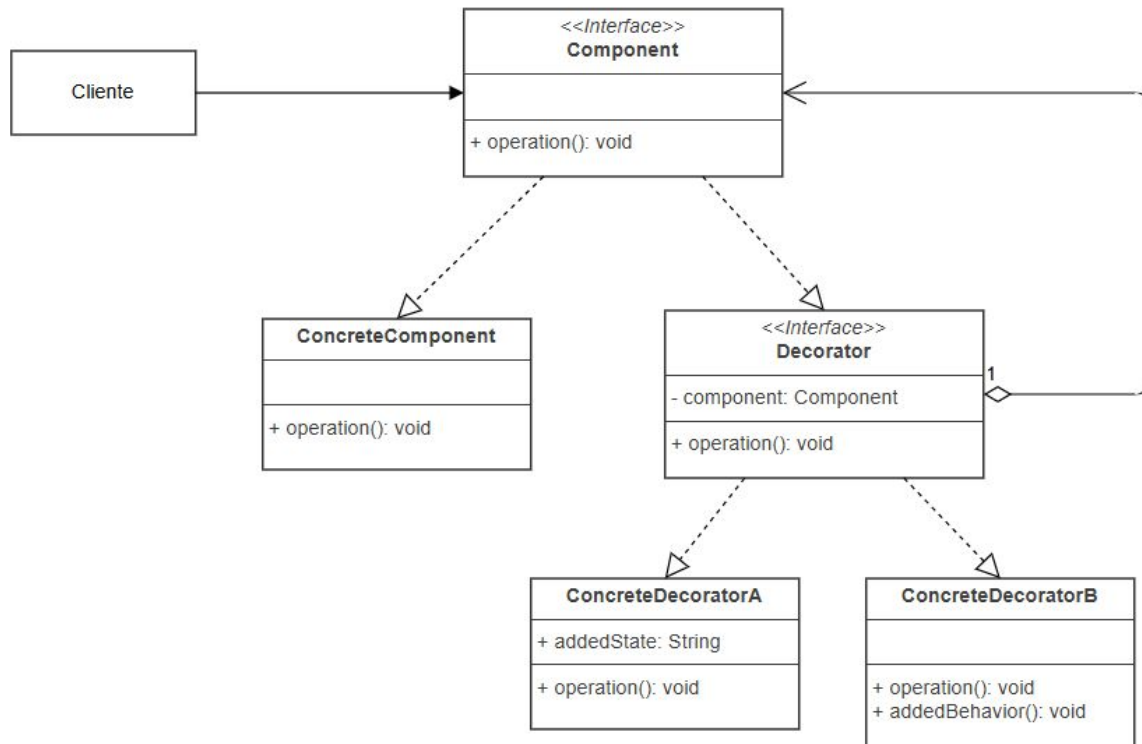
GoF, Padrões de Projetos. p.170



## MOTIVAÇÃO

Em algumas situações, queremos adicionar responsabilidades a objetos individuais, e não em toda a classe. Por exemplo, em um jogo de RPG temos um personagem base que vai incrementando habilidades em cima de seu estado base, acrescentando coisas sem alterar o personagem base.

# DIAGRAMA





## PARTICIPANTES

- **Component:** interface que define uma operação executada no sistema
- **ConcreteComponent:** implementação da operação executada no sistema
- **Decorator:** interface que estende a operação e mantém uma composição com o próprio componente
- **ConcreteDecoratorX:** implementação do decorator, com características próprias, mas mantendo a composição





## PARTICIPANTES

- **Cliente:** elemento que cria a instância do componente e incrementa funcionalidades ao redor do componente concreto



## UTILIZAÇÃO

No cliente, iniciamos a instância do componente concreto. A partir disso, adicionamos implementações do decorador, que irão rodear componente, incrementando funcionalidades ao método executado.

Semelhante ao padrão **Composite**, temos a recursividade de métodos presentes, mas agora apenas com uma referência.

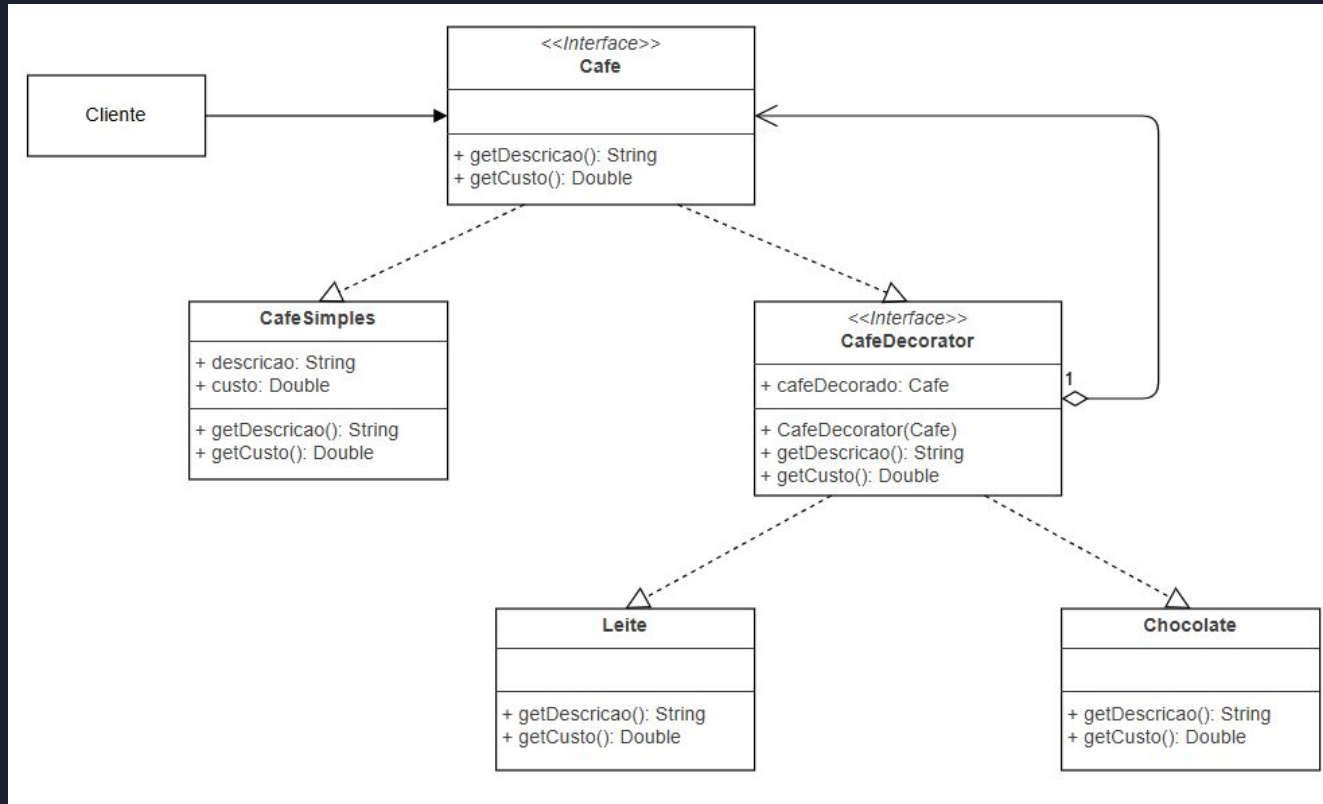


## HORA DE EXERCÍCIO

Vamos tratar de um sistema de personalização de pedidos de uma cafeteria. Nesse sistema, temos um objeto base que representa um café e, na formação do pedido, inserimos adicionais ao café (leite, chocolate, caramelo etc).

Vamos desenvolver esse sistema.

# DIAGRAMA





## HORA DE EXERCÍCIO

Imagine um sistema que envia alertas. O núcleo envia apenas E-mail. Mas o cliente quer receber também por SMS, WhatsApp e Slack.

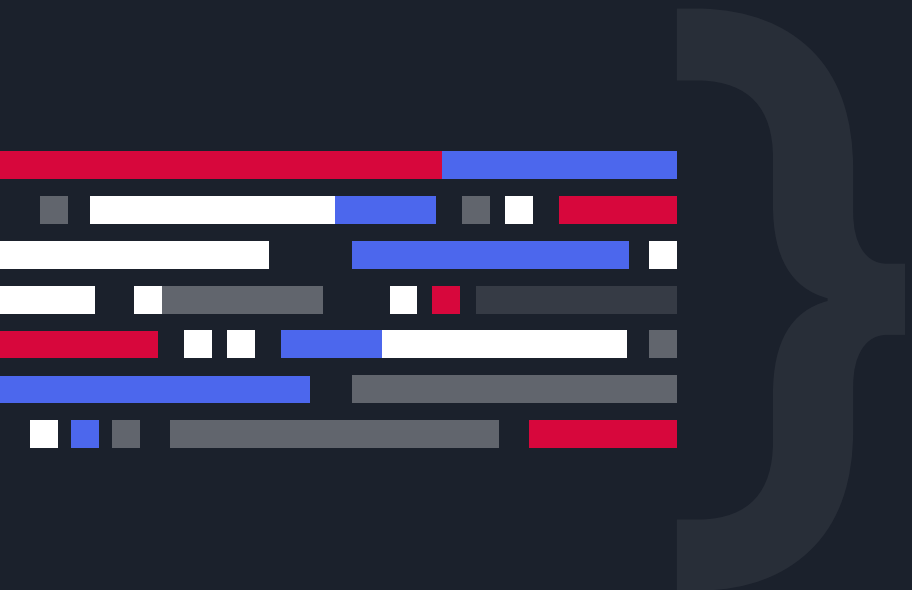
Dessa forma, podemos configurar diferentes combinações de canais de comunicação para enviar alertas junto do e-mail.

**DIAGRAMA?**



## APLICABILIDADE

- Dentro do sistema, precisamos adicionar responsabilidades a objetos individuais de forma dinâmica e transparente
- Algumas funcionalidades podem ser removidas
- No desenvolvimento, a extensão através de subclasses não é prática



## CONSEQUÊNCIAS

- Maior flexibilidade do que a herança estática
- Evita classes sobrecarregadas de características na parte superior da hierarquia
- Um decorator e seu componente não são idênticos
- Grande quantidade de pequenos objetos





FACADE





## DEFINIÇÃO

Fornecer uma interface unificada para um conjunto de interfaces em um subsistema. Façade define uma interface de nível mais alto que torna o subsistema mais fácil de ser usado.

GoF, Padrões de Projetos. p.179

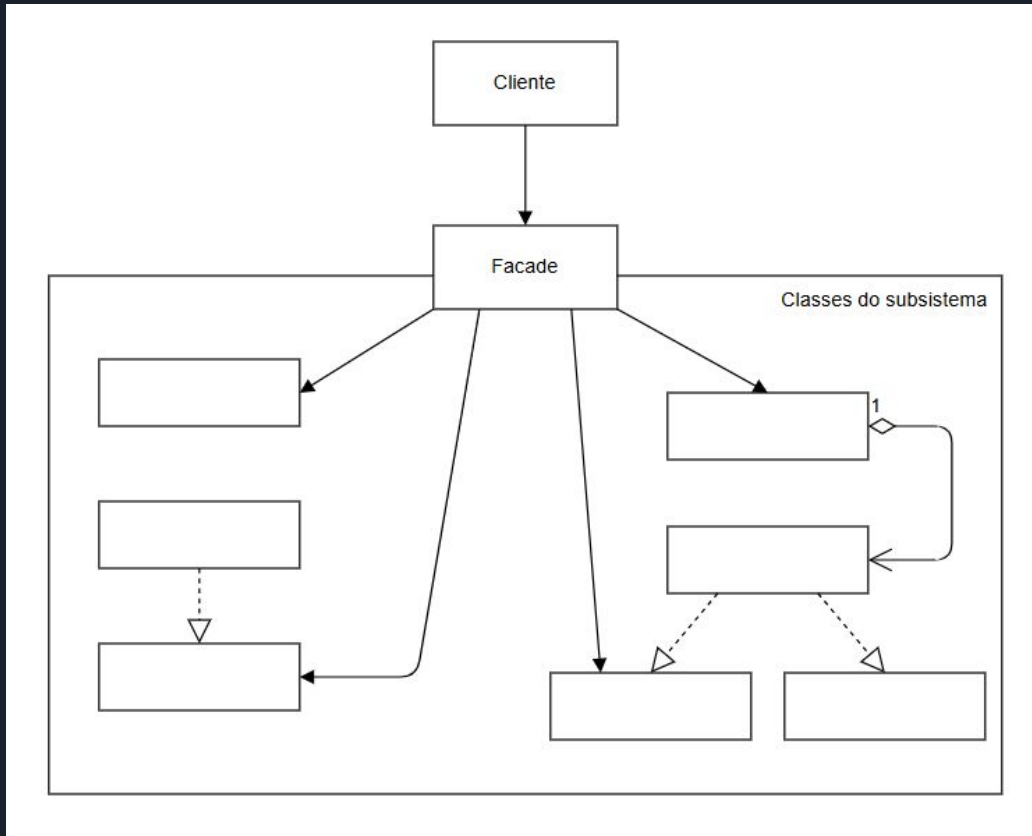


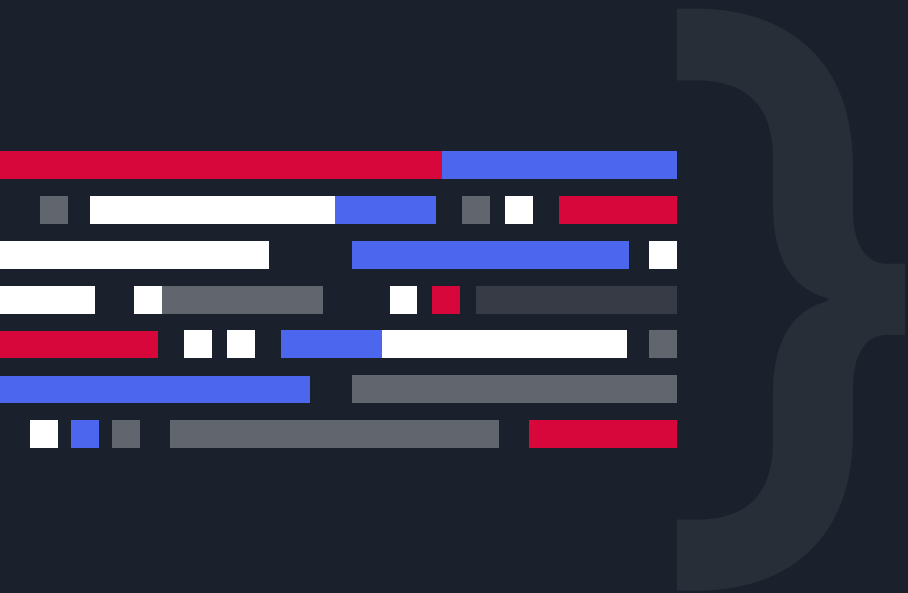
## MOTIVAÇÃO

Dentro de sistemas complexos, temos diferentes componentes de software desempenhando diferentes tarefas, aonde precisamos orquestrar esses componentes para executar regras de negócio e comunicações com fontes externas.

Ao invés de manipular esses componentes de forma individual, podemos oferecer uma fachada que simplifica essa orquestração ao cliente.

# DIAGRAMA





## PARTICIPANTES

- **Façade (Compiler):** classe que conhece todos os componentes que formam o subsistema. Dessa forma, ele oferece um ponto unificado de interação entre os componentes e o cliente
- **Classes do subsistema:** componentes com interfaces e interações complexas que estão encapsuladas pela fachada



## UTILIZAÇÃO

A fachada configura todos os componentes do subsistema necessários para oferecer os pontos de interação. A partir disso, o cliente interage com essas funcionalidades a partir da fachada criada.

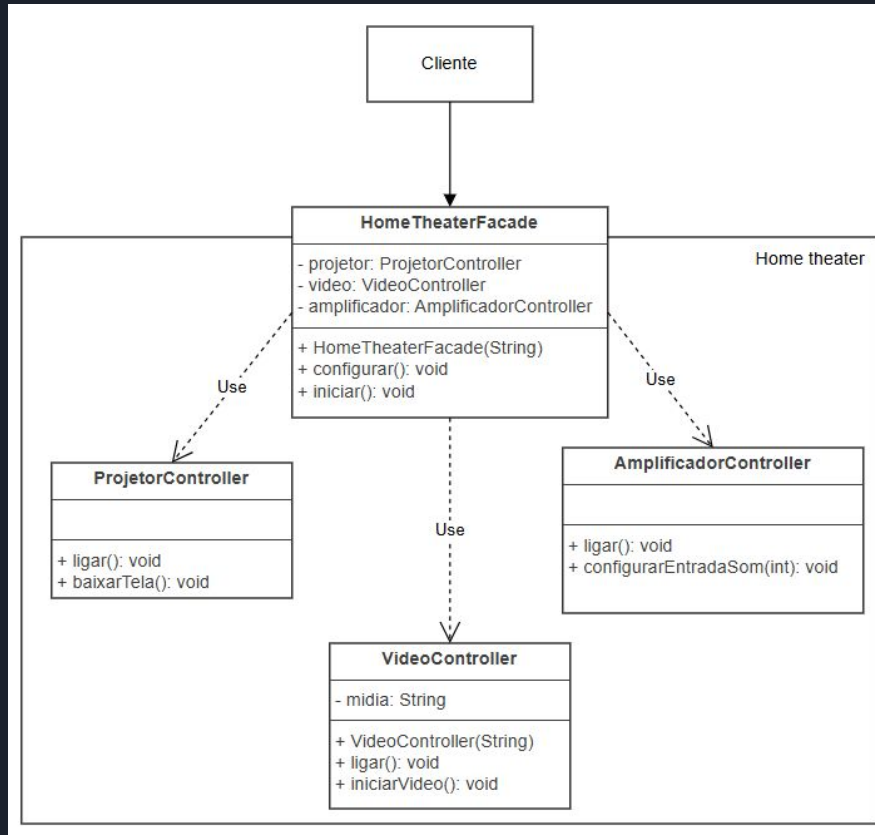


## HORA DE EXERCÍCIO

Para assistir a um filme, você precisaria realizar várias etapas: ligar o projetor, baixar a tela, ligar o amplificador, configurar a entrada de som, ligar o player de vídeo e dar o play.

Sem uma fachada, precisamos fazer isso de forma manual em cada etapa. No entanto, podemos definir uma fachada que orchestra todo esse processo.

# DIAGRAMA

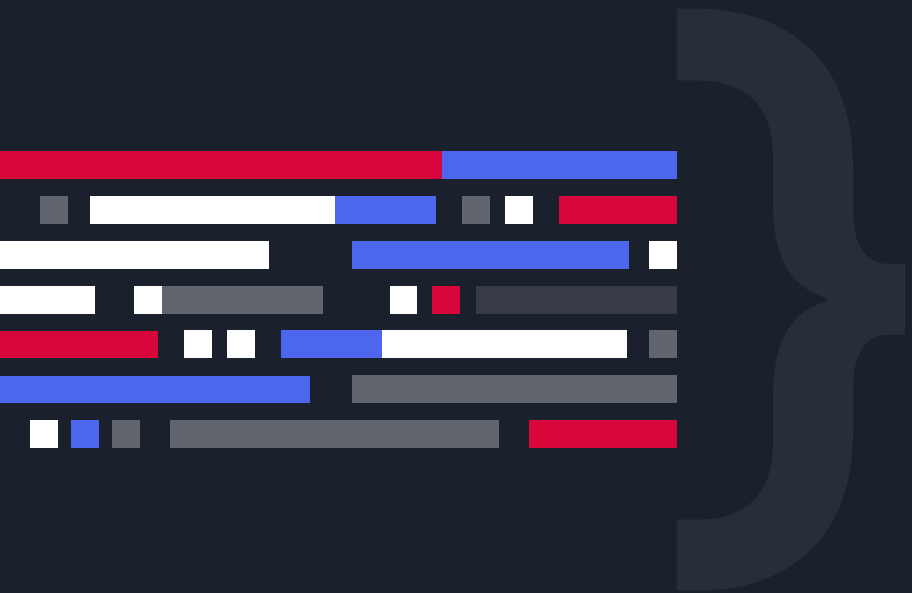






## APLICABILIDADE

- Fornecer uma interface simples de um subsistema complexo
- Há muitas dependências entre clientes e classes de implementação
- Estruturar subsistemas em uma organização de camadas



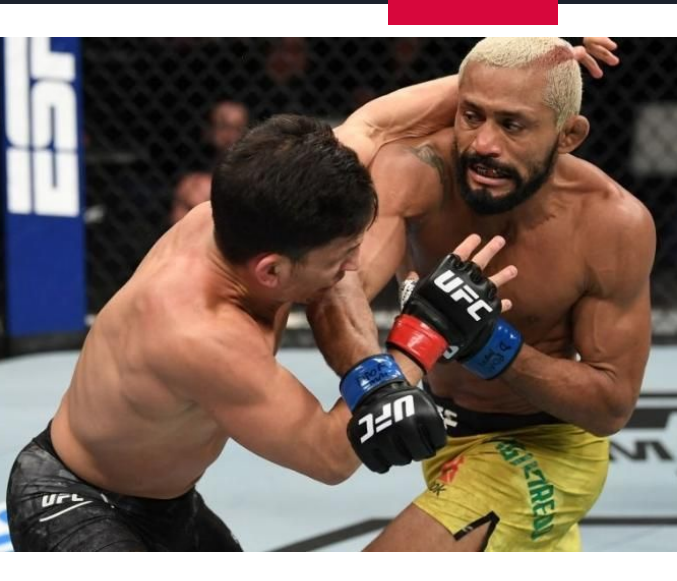
## CONSEQUÊNCIAS

- Isolar clientes dos componentes do subsistema
- Promove acoplamento fraco entre o subsistema e seus clientes
- Não impede as aplicações de utilizarem os componentes do subsistema, quando necessário



# FLYWEIGHT

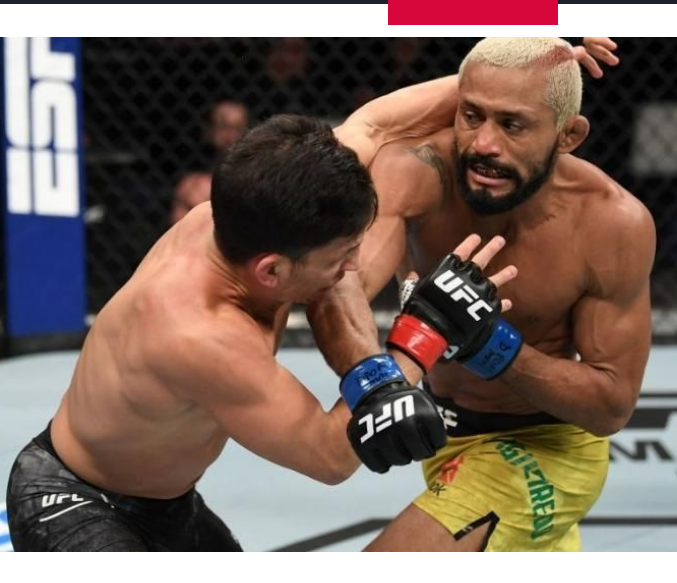




## DEFINIÇÃO

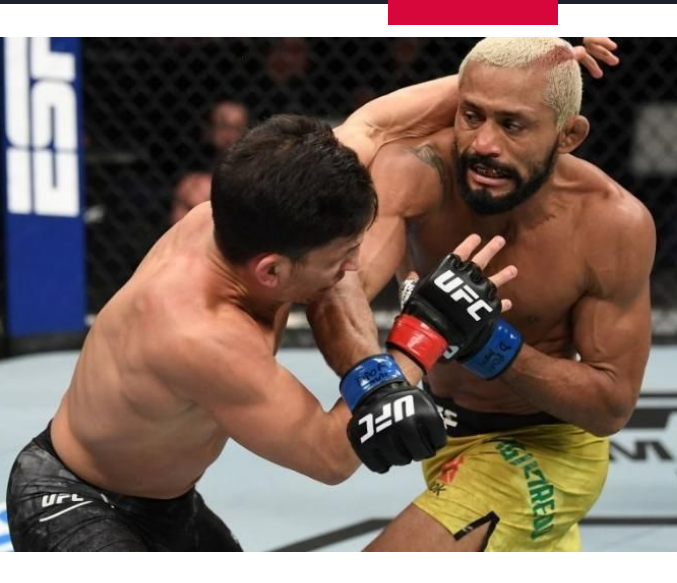
Usar compartilhamento para suportar eficientemente grandes quantidades de objetos de granularidade fina.

GoF, Padrões de Projetos. p.187



## MOTIVAÇÃO

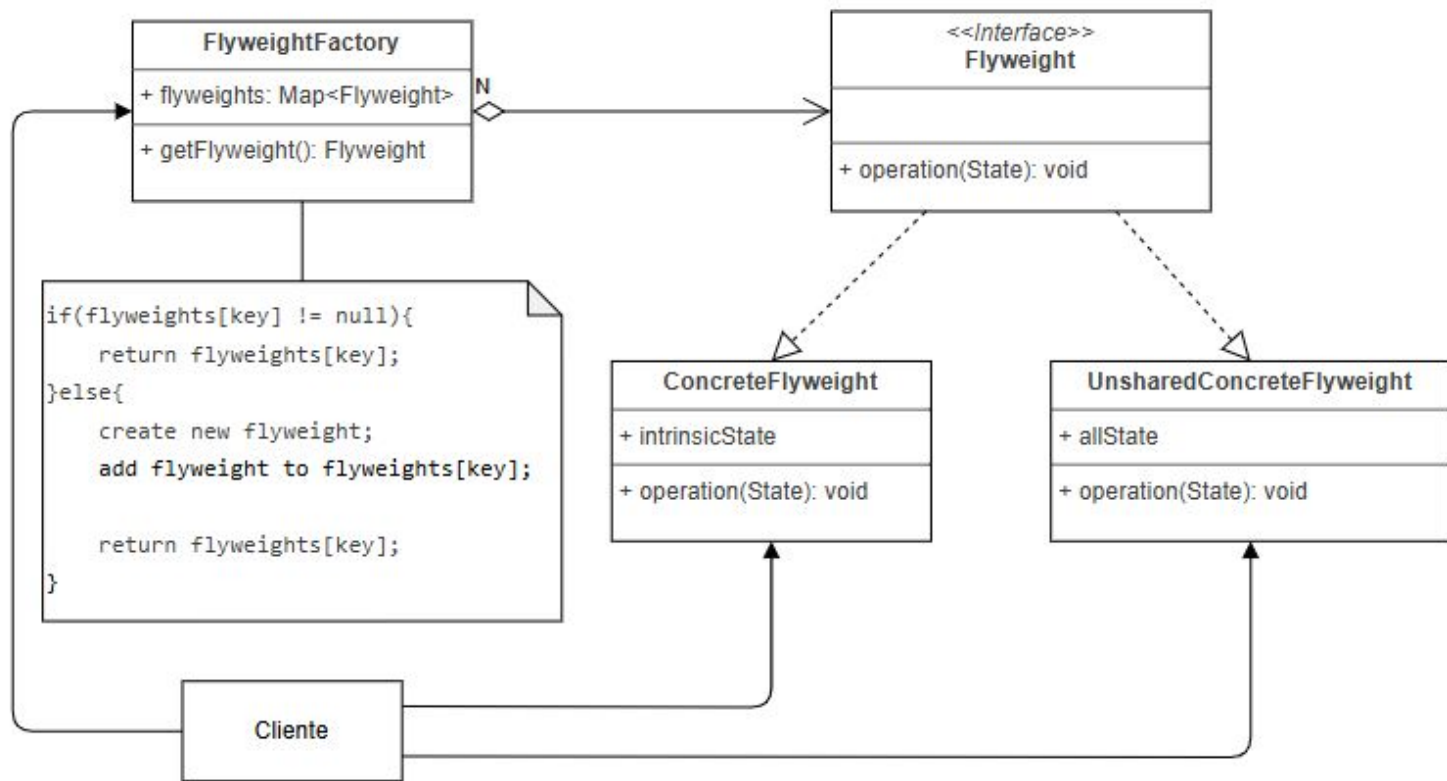
Imagine que você precisa criar 100.000 objetos em um sistema. Se cada objeto carregar todas as suas informações internas, a memória (RAM) pode esgotar rapidamente. O Flyweight resolve isso dividindo os dados do objeto em dois tipos.



# MOTIVAÇÃO

- **Estado Intrínseco:** Informações que são constantes, repetitivas e podem ser compartilhadas (ex: a cor ou a textura de um elemento que se repete).
- **Estado Extrínseco:** Informações que variam de acordo com o contexto e não podem ser compartilhadas (ex: a coordenada (x, y) de onde o objeto está posicionado).

# DIAGRAMA

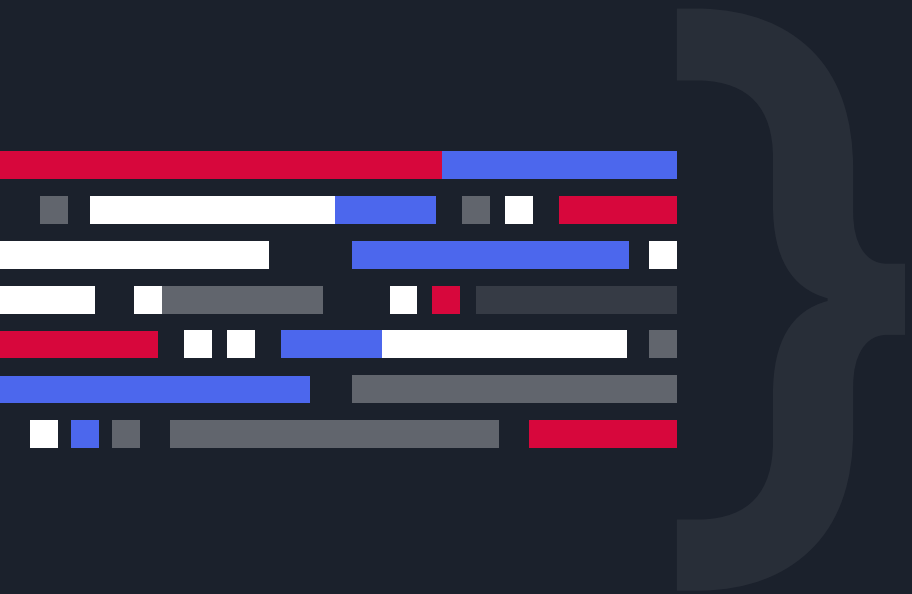




## PARTICIPANTES

- **Flyweight:** interface que define um objeto que pode receber e atuar sobre estados extrínsecos
- **ConcreteFlyweight:** implementação do objeto que acrescenta armazenamento de estados intrínsecos, se houver. Um objeto ConcreteFlyweight deve ser compartilhável
- **UnsharedConcreteFlyweight:** implementação do objeto que não necessita compartilhar objetos Flyweight





## PARTICIPANTES

- **FlyweightFactory:** classe que cria e gerencia objetos Flyweight, garantindo que os objetos sejam compartilhados de forma adequada
- **Cliente:** elemento que mantém referência para objetos Flyweight, computando ou armazenando o estado extrínseco dos objetos



## UTILIZAÇÃO

O `FlyweightFactory` é responsável por criar (se não existir) e retornar objetos `Flyweight` com estado intrínseco. No cliente, utilizamos dos estados extrínsecos para computar os objetos intrínsecos e aplicar as operações que manipulam os estados na aplicação.

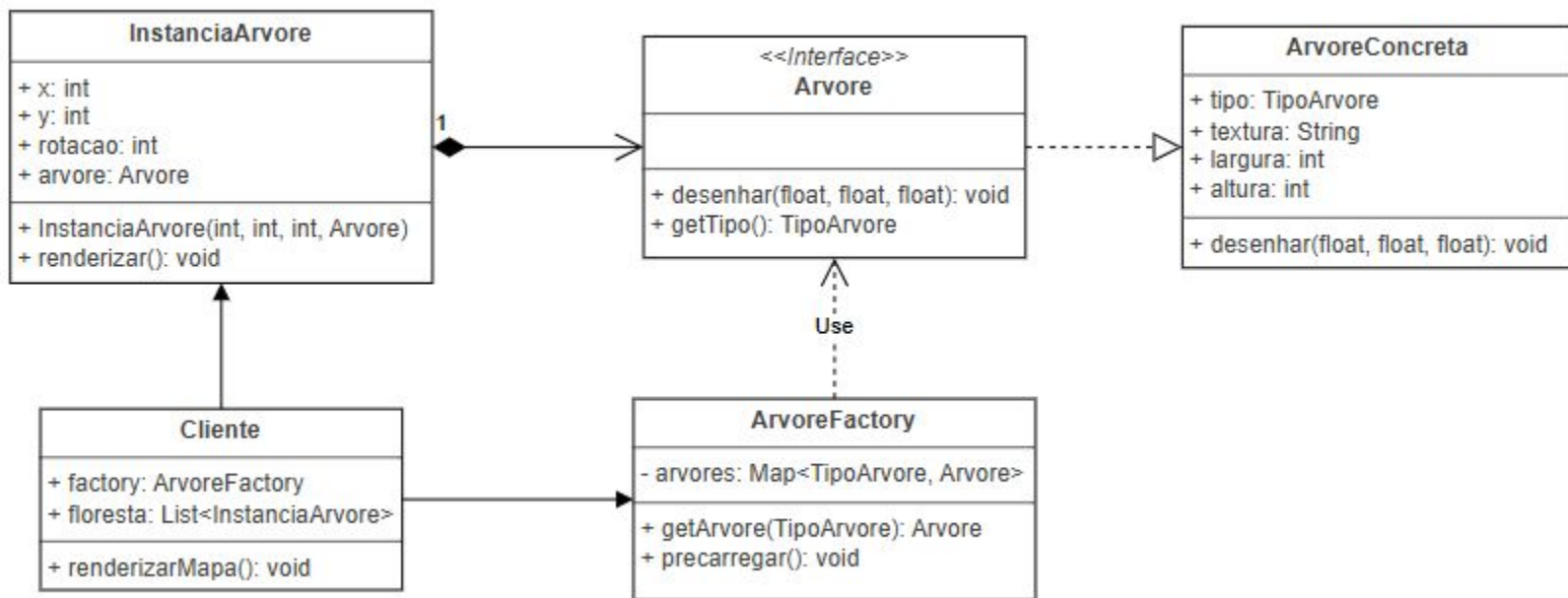


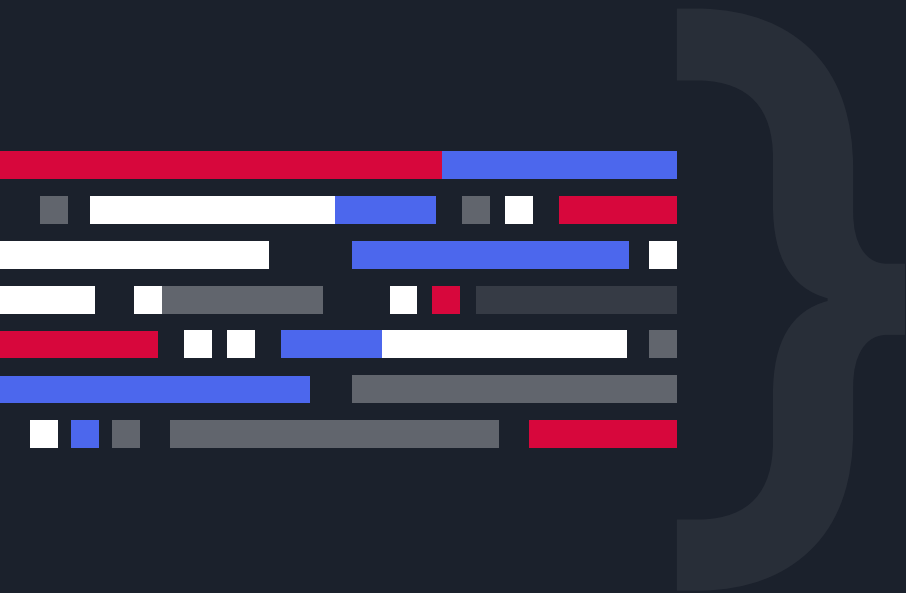
## HORA DE EXERCÍCIO

Em um jogo, temos elementos de cenário que é composto por árvores de alguns tipos previstos para renderização. Temos árvores sendo renderizadas em diferentes posições do mapa, mas sempre apresentando as mesmas características entre si, para árvores de um mesmo tipo.

Vamos implementar esse sistema de mapas.

# DIAGRAMA





## APLICABILIDADE

- Uma aplicação que utiliza um grande número de objetos, causando altos custos de armazenamento
- A maioria dos estados dos objetos pode ser extrínseco
- A aplicação não depende da identidade dos objetos



## CONSEQUÊNCIAS

- Economia de armazenamento sendo uma junção de vários fatores:
  - Reduzir o número total de instâncias obtidas com o compartilhamento
  - A quantidade de estados intrínsecos
  - O estado extrínseco é computado ou armazenado



PROXY





## DEFINIÇÃO

Fornecer um substituto (surrogate) ou marcador da localização de outro objeto para controlar o acesso a esse objeto.

GoF, Padrões de Projetos. p.198



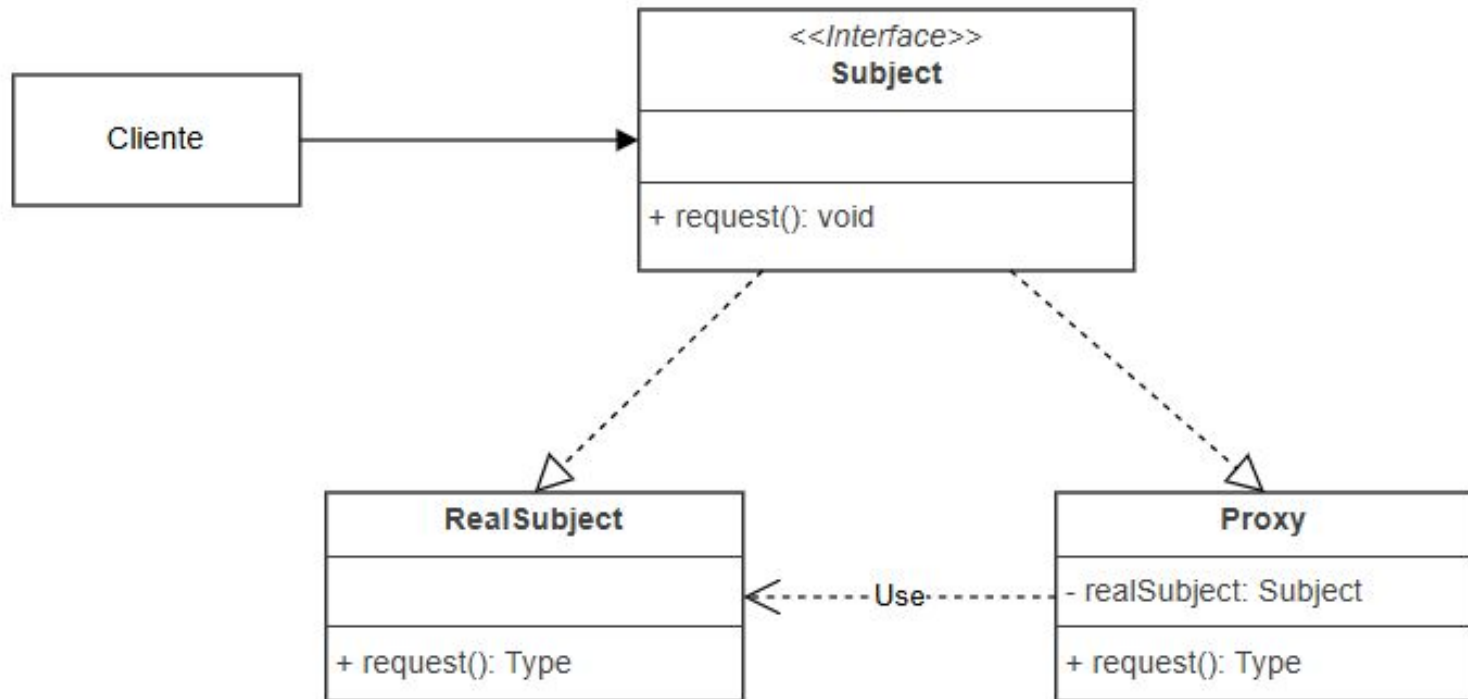


## MOTIVAÇÃO

A ideia de controlar o acesso a um objeto gira em torno de adiar o custo integral da criação e inicialização até o instante em que é realmente necessário utilizá-lo.

Por exemplo, imagine o custo envolvido para carregar uma imagem do sistema de arquivos para renderização em um e-commerce.

# DIAGRAMA





## PARTICIPANTES

- **Subject:** interface que mantém relação entre RealSubject e Proxy
- **RealSubject:** classe que define o objeto real que o proxy representa
- **Proxy:** classe que usa a interface definida por Subject, mantendo uma referência que permite ao proxy usar o objeto real
- **Cliente:** elemento que interage com o proxy para fazer uso do objeto gerenciado



## UTILIZAÇÃO

No cliente, fazemos uso do proxy para acessar o objeto gerenciado. Internamente, esse objeto é manipulado com a mesma interface utilizada pelo proxy. Dessa forma, aonde for necessário utilizar o objeto real, podemos fazer uso do proxy sem alterações na operação.



## TIPOS DE PROXY

- **Proxy virtual:** O proxy mantém uma referência leve do objeto. Somente quando ele é necessário, seu carregamento é finalizado
- **Proxy de proteção:** O proxy valida a autorização de uso do objeto real
- **Proxy remoto:** O proxy empacota os detalhes de interação com um objeto que está em um espaço diferente

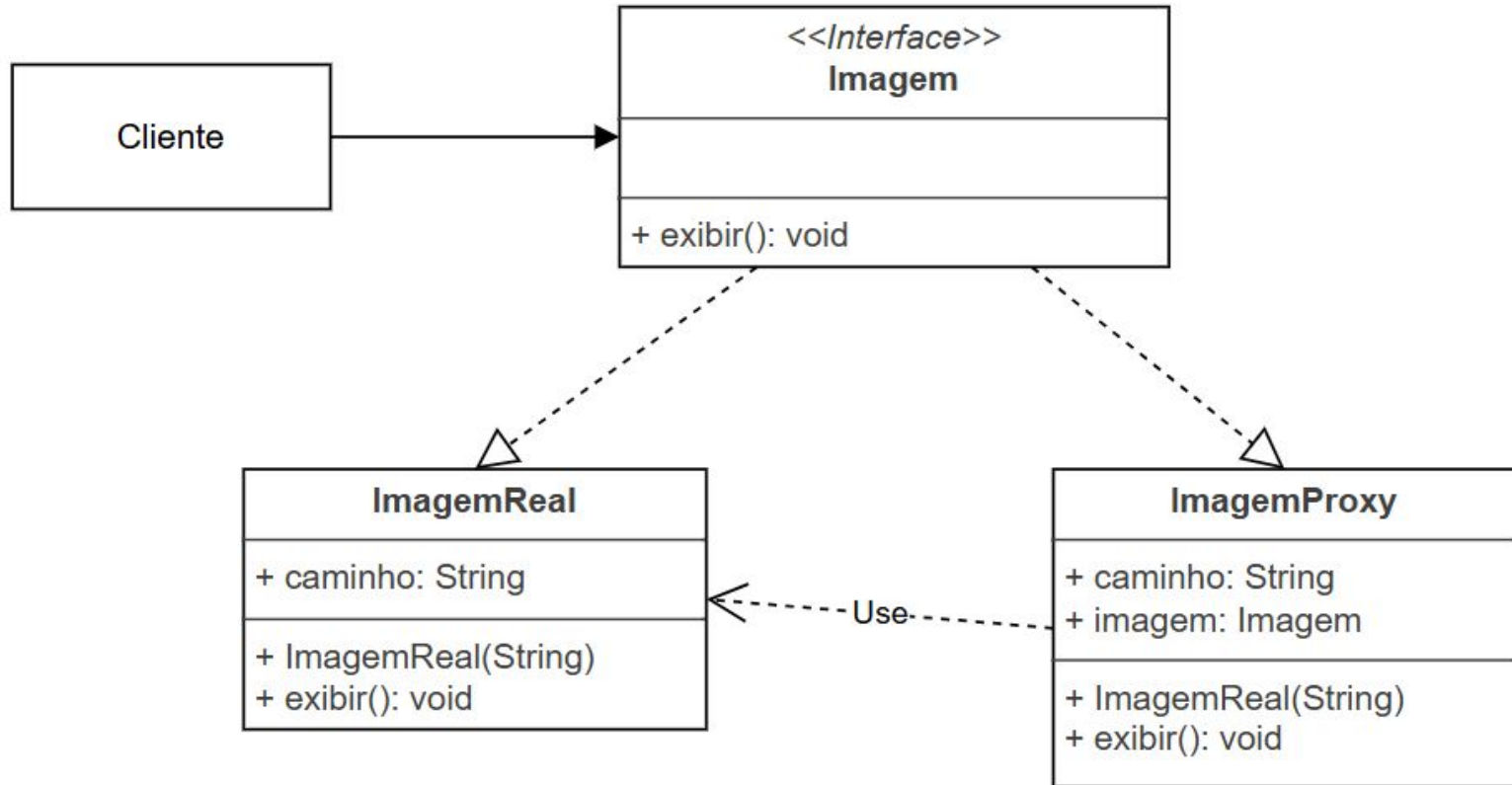


## HORA DE EXERCÍCIO

Imagine um sistema de e-commerce que exibe uma lista de 1.000 produtos. Cada produto possui uma imagem em alta resolução. Carregar todas as 1.000 imagens na memória assim que a página abre deixaria a aplicação lenta e consumiria gigabytes de RAM.

Vamos implementar essa renderização das imagens.

# DIAGRAMA





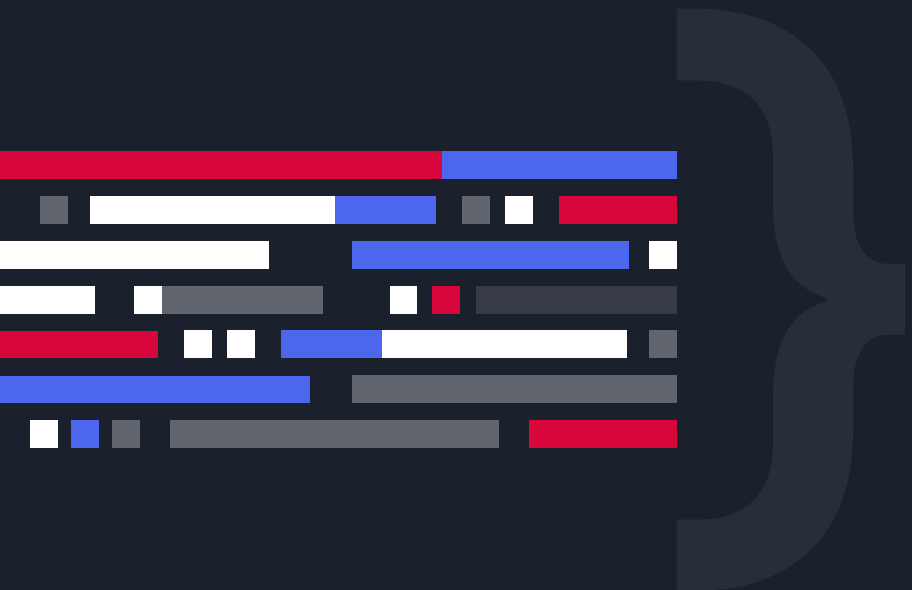
## HORA DE EXERCÍCIO

Em um sistema hospitalar, o prontuário do paciente pode ter seu histórico visualizado e ter diagnósticos alterados. No entanto, devo limitar essas operações somente às pessoas da enfermagem, e os demais usuários não devem ser capazes de realizar essas operações.

Implemente este sistema.



**DIAGRAMA?**



## APLICABILIDADE

- A aplicação precisa manter representação local de objetos em outros ambientes
- A aplicação precisa carregar objetos sob demanda
- A aplicação precisa controlar o acesso a determinados objetos



## CONSEQUÊNCIAS

- Um proxy remoto oculta o fato do objeto estar em outro ambiente
- Um proxy pode executar otimizações para criação de objetos sob demanda
- Podemos ter tarefas relacionadas a validação de acesso a objetos do sistema

# PADRÕES ESTRUTURAIS

Ao longo dos exemplos, estratégias de estruturação dos objetos, sempre focando em atender um requisito específico da formação de objetos complexos dentro do sistema.

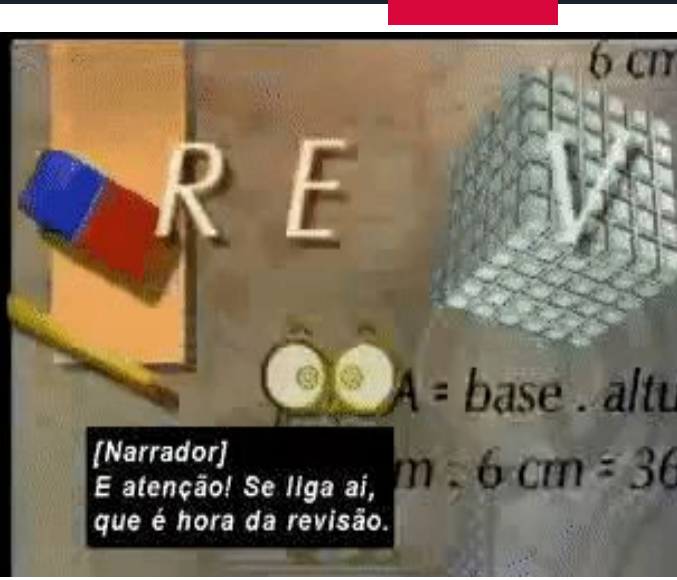
Tal qual os padrões criacionais, a combinação de padrões permite soluções mais robustas e alinhadas com o que o negócio espera.





# PADRÕES

- **Adapter:** adaptar objetos de assinaturas diferentes
  - **Bridge:** conectar duas partes da aplicação sem gerar dependência explícita
  - **Composite:** enriquecer objetos com uma estrutura alinhada de objetos individuais e composições
  - **Decorator:** adicionar novas funcionalidades seguindo um mesmo contrato
  - **Façade:** esconder detalhes de orquestração de objetos, oferecendo um ponto único de interação
  - **Flyweight:** gerenciar grandes quantidades de objetos com estados compartilhados
  - **Proxy:** gerenciar o acesso a objetos pesados
- 



## RECAPITULANDO

- Padrão Decorator
- Padrão Façade
- Padrão Flyweight
- Padrão Proxy
- Revisão dos padrões de projeto estruturais

# OBRIGADO!



**"AQUI DO LADO PERDENEIRAS..."**

**Lasier Martins**